

Almost all algorithms use a technique that involves the gradual filling in of a table. In common programming languages, this is done by looping over the table indices. This is not possible in SQL (at least not as easily). So how do we approach such problems?

We somehow have to find the indices for the next element. What I always did: I joined the input table (like the input strings table in lcs or the value-weight-table in knapsack) with the recurring table and looked for two things:

1. Which elements have an upper and a lower neighbor?
2. Which element is not yet defined in the recurring table?

This element is always unique and the next element to focus on.

For some reason, this technique turned out to be faster than calculating the indices manually. I don't know why and it seems odd. Now, at my third approach at such a problem (knapsack), I want to know if there is a better way (there has to be)

I mean.. can't I just use the indices of the previous iteration? This HAS to be faster

Alright, yeah, I still have to use the recurring table of course, because I still need those values. In contrast to LCS though, in knapsack, I cannot use the technique mentioned earlier as easily, because the values that I need to calculate the new value exist for various values. Woah, wait. Why wait? If the values already exist, why not compute them already?

Ok, so we can calculate an entire row in one iteration. But this only works because we don't need any results from the current row. Update: It works!

Why USING-KEY might be useful in knapsack

...well I mean, we don't need all those intermediate values later on. So why not just overwrite them?